

Grafos

Prof. Dr. Matheus da Silva Menezes



Unidade 03

GRAFOS

GRAFOS

Ciclo Hamiltoniano e PCV

The “icosian game”

Foi inventado em 1856 pelo matemático William Rowan Hamilton

Idéia Geral:

O jogo consistia em um tabuleiro em forma de dodecaedro regular (um sólido com 12 faces pentagonais e 20 vértices). Cada vértice representa uma cidade e as arestas representam rotas diretas entre cidades vizinhas.

O desafio proposto era o seguinte:

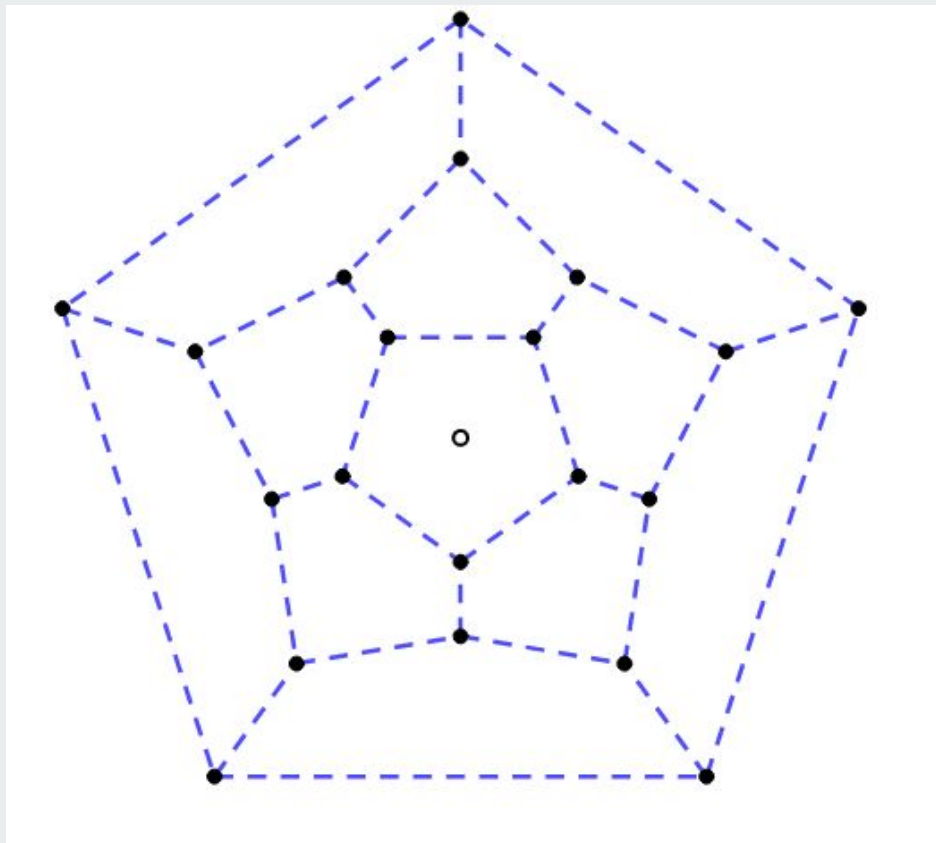
Encontrar um trajeto fechado que passe uma única vez por todas as cidades e retorne à cidade de origem.

Em outras palavras:

- *O jogador deveria visitar todas as 20 cidades exatamente uma vez;*
- *E ao final, voltar à cidade inicial, formando um circuito contínuo;*
- *As viagens só poderiam seguir pelas conexões permitidas entre cidades adjacentes do dodecaedro (ou seja, pelas bordas do sólido).*



The “icosian game”



Definição

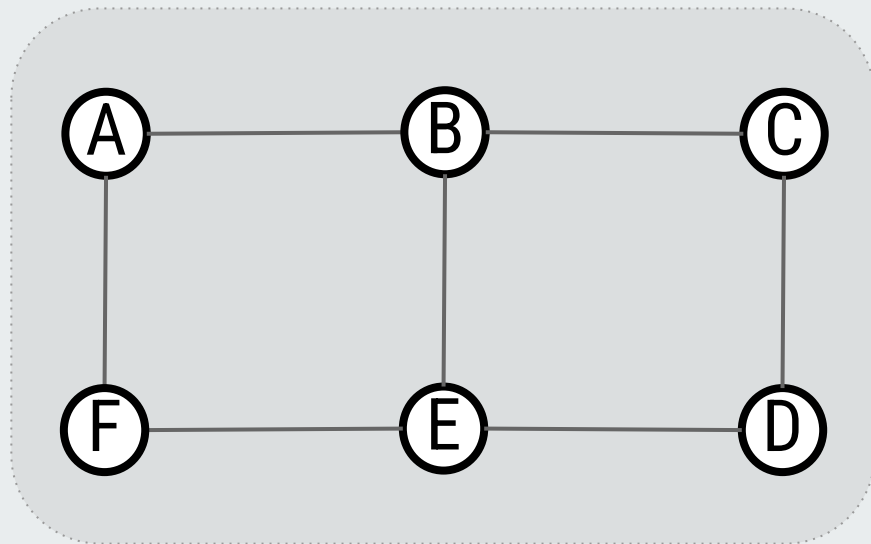
Dado um grafo $G = (V, A)$, um **ciclo** H que passa por todos os vértices de G é dito **hamiltoniano**.

G é dito **hamiltoniano** se possui **ciclo hamiltoniano**.

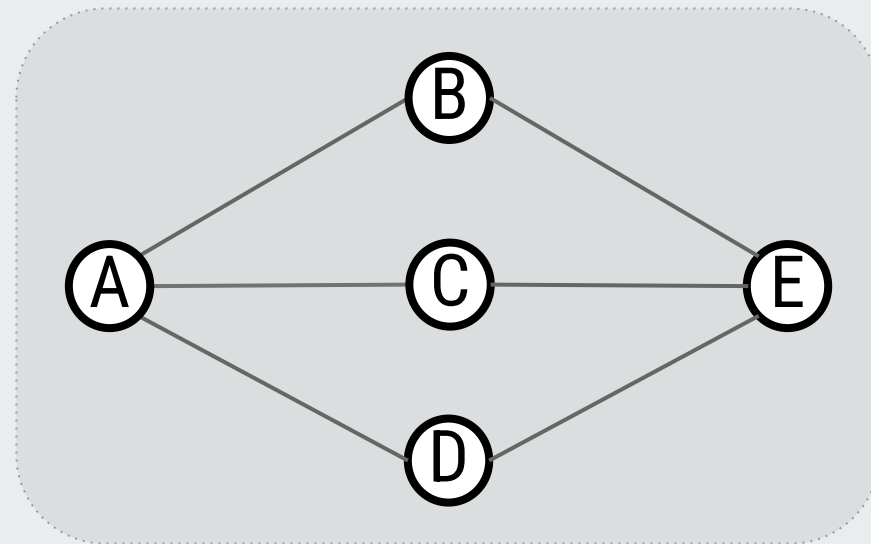
Definição

Dado um grafo $G = (V, A)$, um **caminho** C que passa por todos os vértices de G é dito **semi-hamiltoniano**.

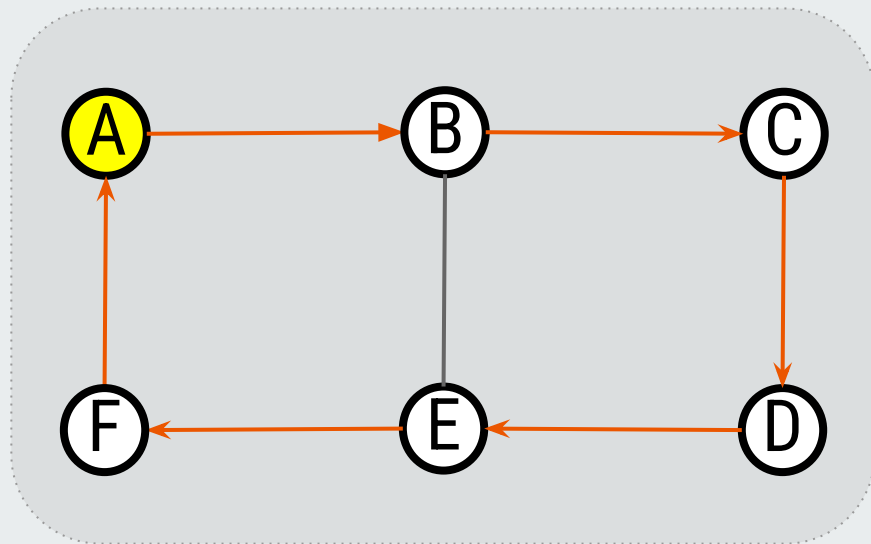
G é dito **semi-hamiltoniano** se possui **caminho hamiltoniano**.



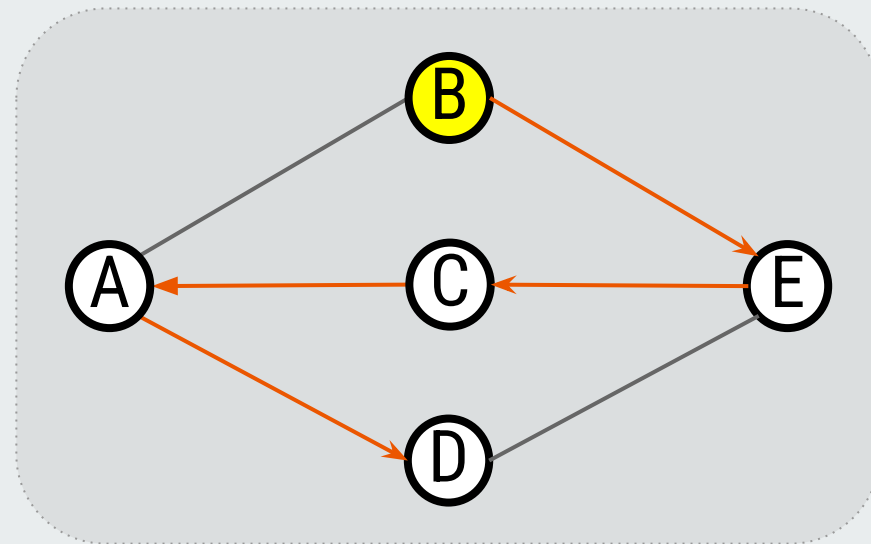
Grafo Hamiltoniano



Grafo Semi-Hamiltoniano



Grafo Hamiltoniano



Grafo Semi-Hamiltoniano

G é hamiltoniano?

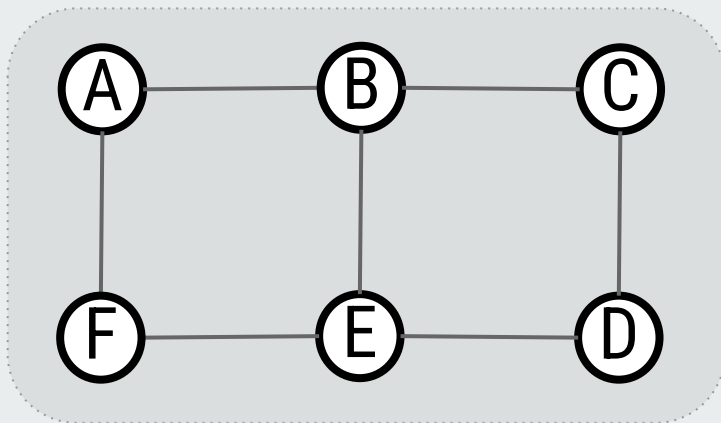
G é semi-hamiltoniano?

são **NP-completos**?

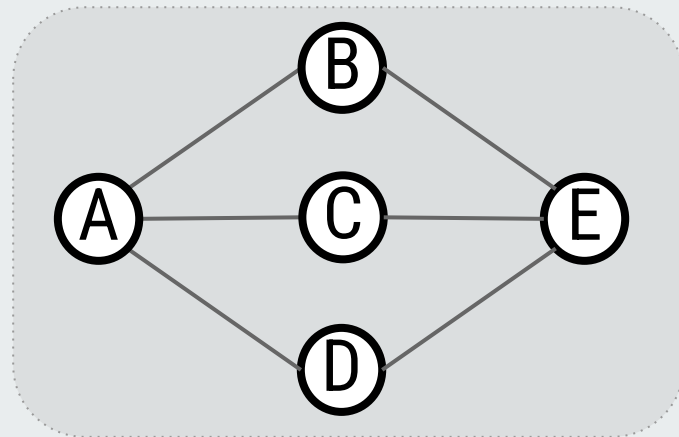
Teorema

Teorema de Dirac (1952)

Um grafo $G = (V, A)$, com n vértices, $n \geq 3$ é hamiltoniano se todo vértice tiver grau maior ou igual a $n/2$.



$$n=6 \rightarrow n/2 = 3$$

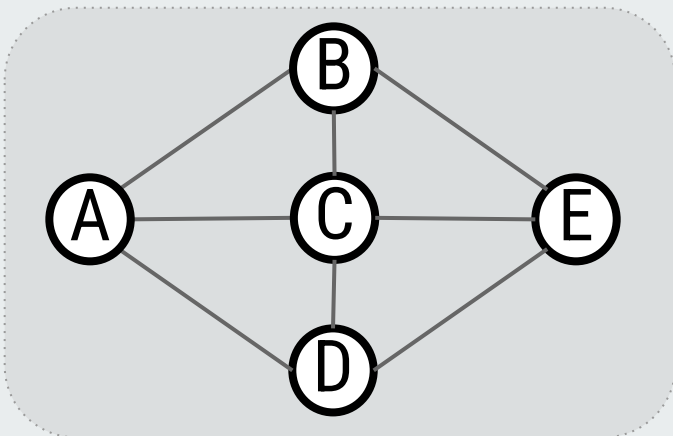


$$n=5 \rightarrow n/2 = 2,5$$
$$3 \geq n/2$$

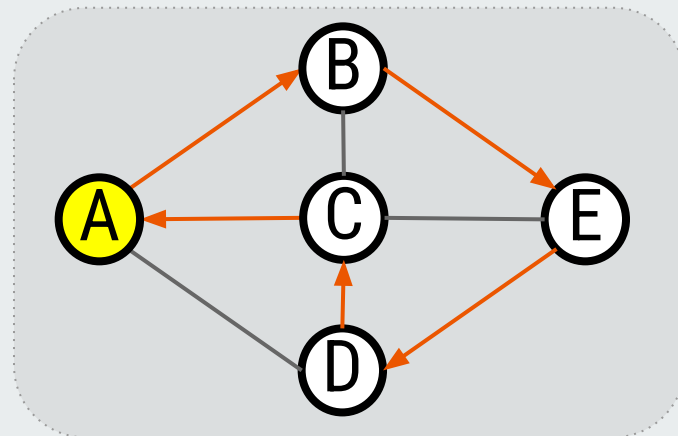
Teorema

Teorema de Dirac (1952)

Um grafo $G = (V, A)$, com n vértices, $n \geq 3$ é hamiltoniano se todo vértice tiver grau maior ou igual a $n/2$.



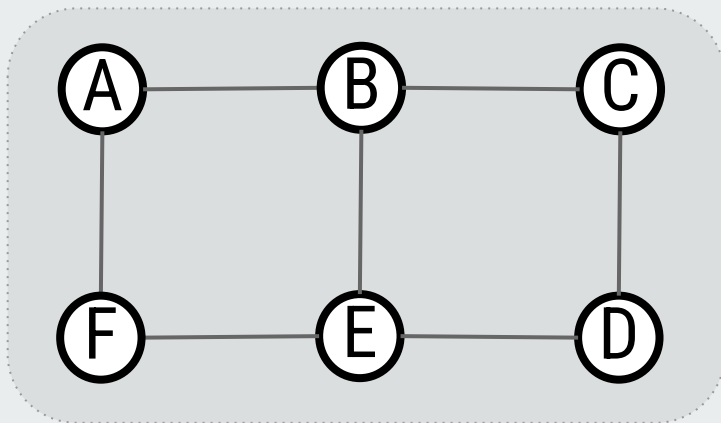
$$n=5 \rightarrow n/2 = 2,5$$
$$3 \geq n/2$$



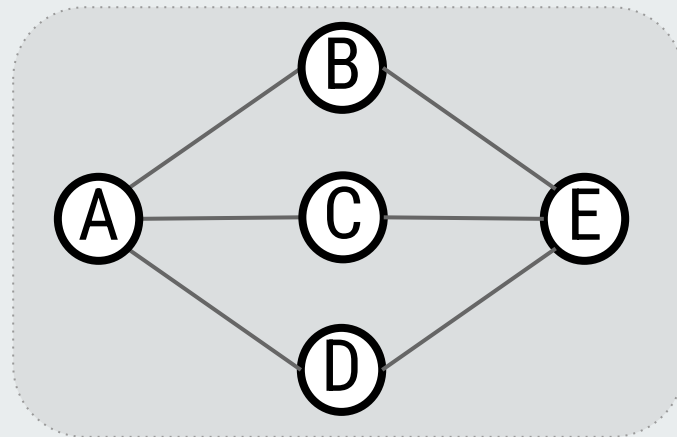
Teorema

Teorema de Ore (1961)

Uma condição suficiente para que um grafo $G = (V, A)$, com n vértices seja hamiltoniano é que a soma dos graus de cada par de vértices não adjacentes seja no mínimo n .



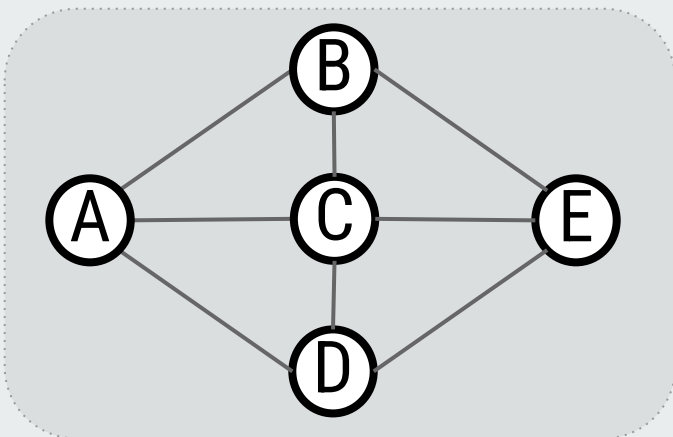
$$A + C = 2 + 2 = 4$$



Teorema

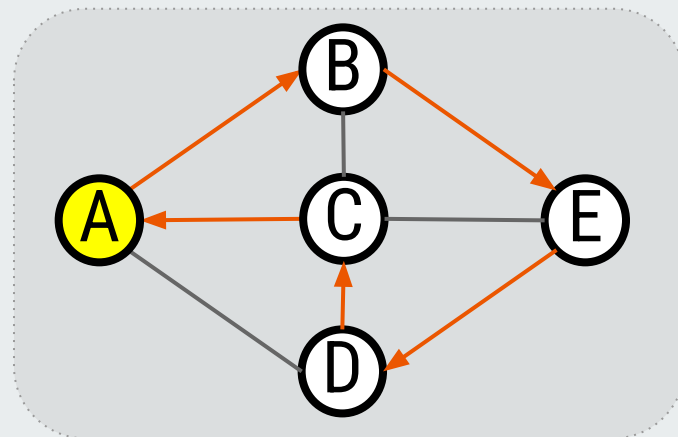
Teorema de Ore (1961)

Uma condição suficiente para que um grafo $G = (V, A)$, com n vértices seja hamiltoniano é que a soma dos graus de cada par de vértices não adjacentes seja no mínimo n .



$$A + E = 3 + 3 = 6 \geq 5$$

$$B + D = 3 + 3 = 6 \geq 5$$



Teorema

Teorema de Chvátal (1972)

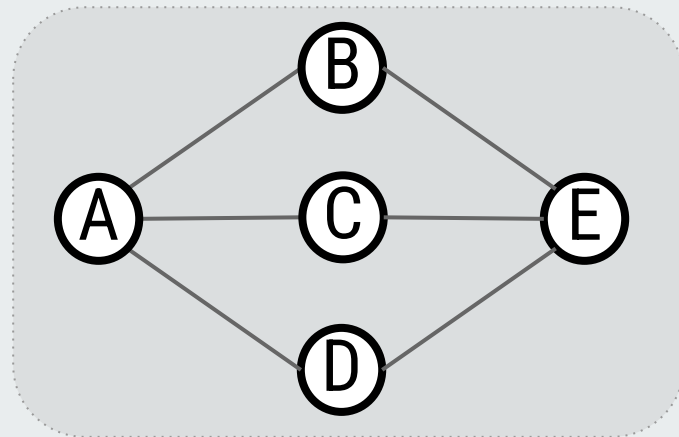
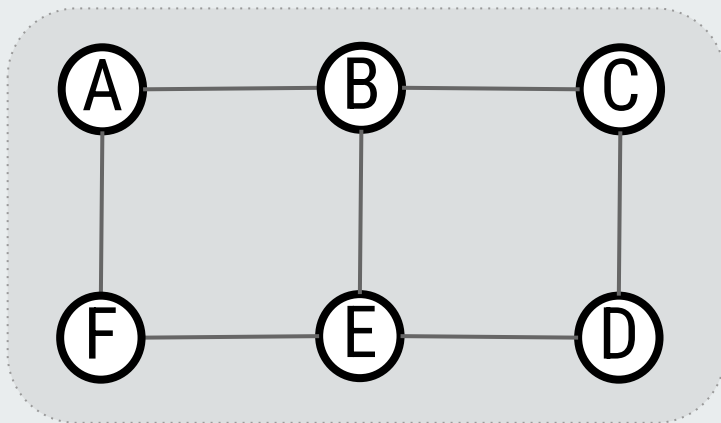
Se um grafo $G = (V, A)$, com n vértices, $n \geq 3$ e sequência de graus ordenada

$$d_1 \leq d_2 \leq \dots \leq d_n,$$

se para todo k , $1 \leq k < n/2$, vale

$$d_{n-k} \geq n-k \text{ ou } d_k > k,$$

então G é hamiltoniano.



Teorema

Teorema de Chvátal (1972)

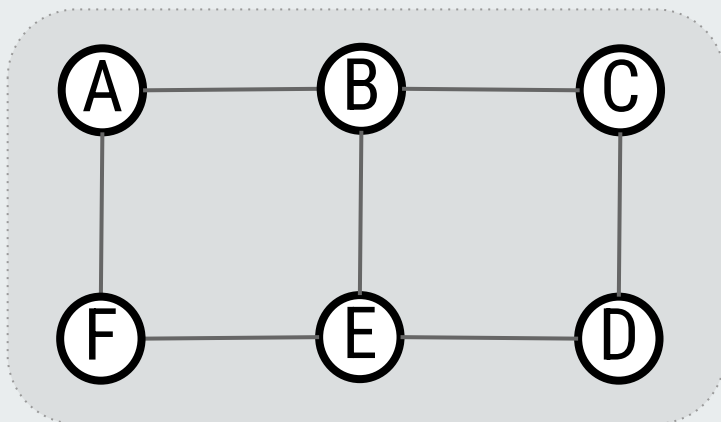
Se um grafo $G = (V, A)$, com n vértices, $n \geq 3$ e sequência de graus ordenada

$$d_1 \leq d_2 \leq \dots \leq d_n,$$

se para todo k , $1 \leq k < n/2$, vale

$$d_{n-k} \geq n-k \text{ ou } d_k > k,$$

então G é hamiltoniano.



$$n = 6$$

Graus: 2, 2, 2, 2, 3, 3

$$k = 1, 2 \quad (\text{pois } n/2=3 \text{ e } k < n/2)$$

$$k=1 \rightarrow d_5 \geq 5 \text{ ou } d_1 > 1 \rightarrow V$$

$$k=2 \rightarrow d_4 \geq 4 \text{ ou } d_2 > 2 \rightarrow F$$

Teorema

Teorema de Chvátal (1972)

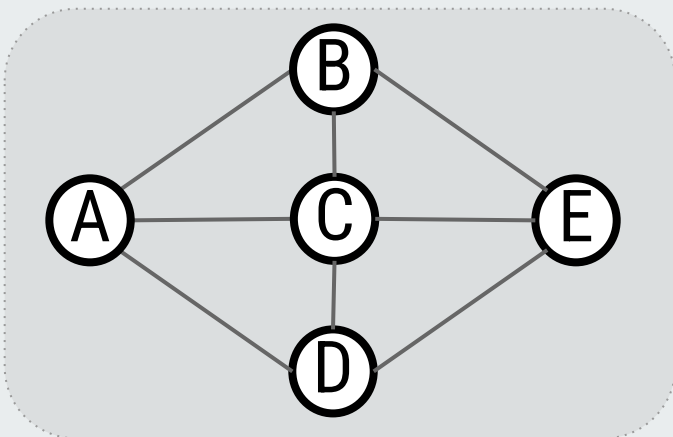
Se um grafo $G = (V, A)$, com n vértices, $n \geq 3$ e sequência de graus ordenada

$$d_1 \leq d_2 \leq \dots \leq d_n,$$

se para todo k , $1 \leq k < n/2$, vale

$$d_{n-k} \geq n-k \text{ ou } d_k > k,$$

então G é hamiltoniano.



$$n = 5$$

Graus: 3, 3, 3, 3, 4

$$k = 1, 2 \quad (\text{pois } n/2=2,5 \text{ e } k < n/2)$$

$$k=1 \rightarrow d_4 \geq 4 \text{ ou } d_1 > 1 \rightarrow V$$

$$k=2 \rightarrow d_3 \geq 3 \text{ ou } d_2 > 2 \rightarrow V$$

Definição

O Problema do Caixeiro Viajante (Traveling Salesman Problem)

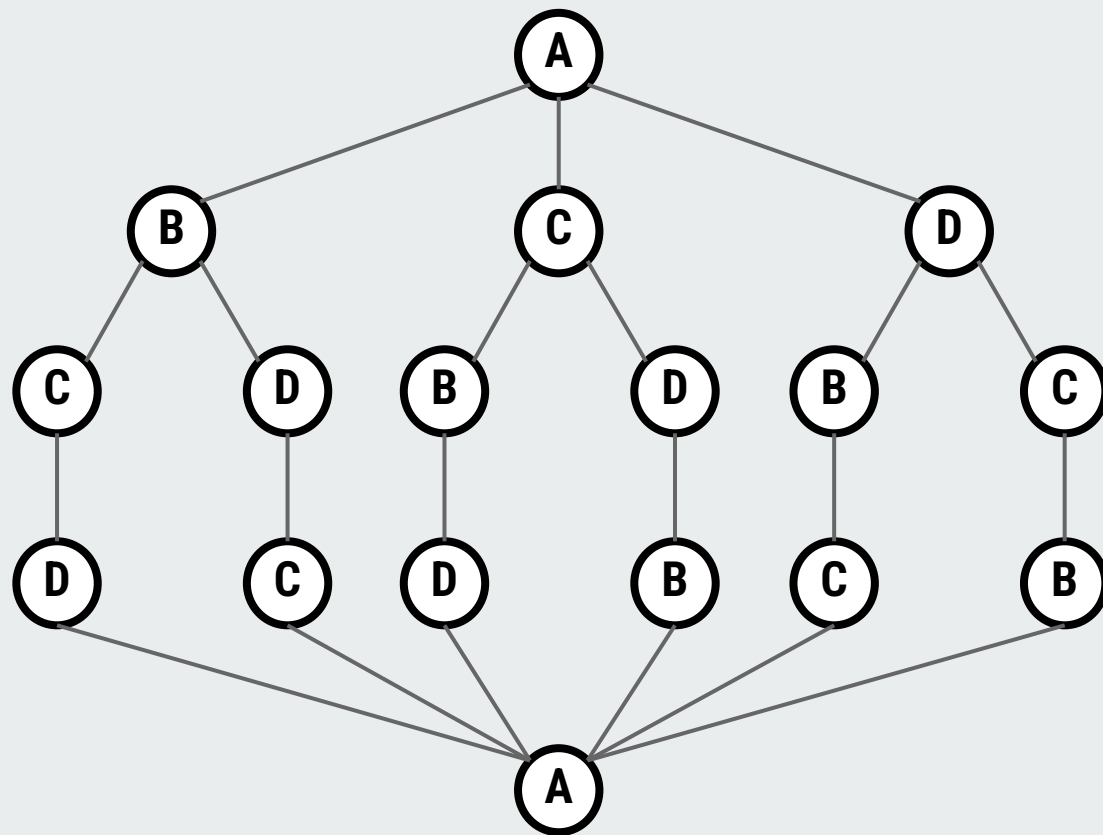
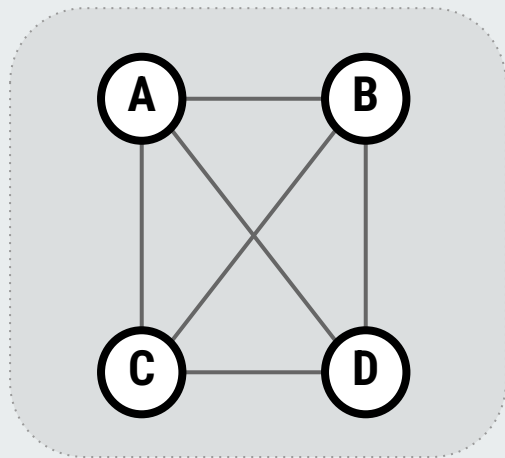
*Dado um grafo ponderado $G = (V, A)$, encontrar um **ciclo** H cuja soma das arestas seja mínima dentre todos os ciclos geradores de G .*

"Dado N cidades, achar a caminho mais curto passando por todas as cidades uma única vez."

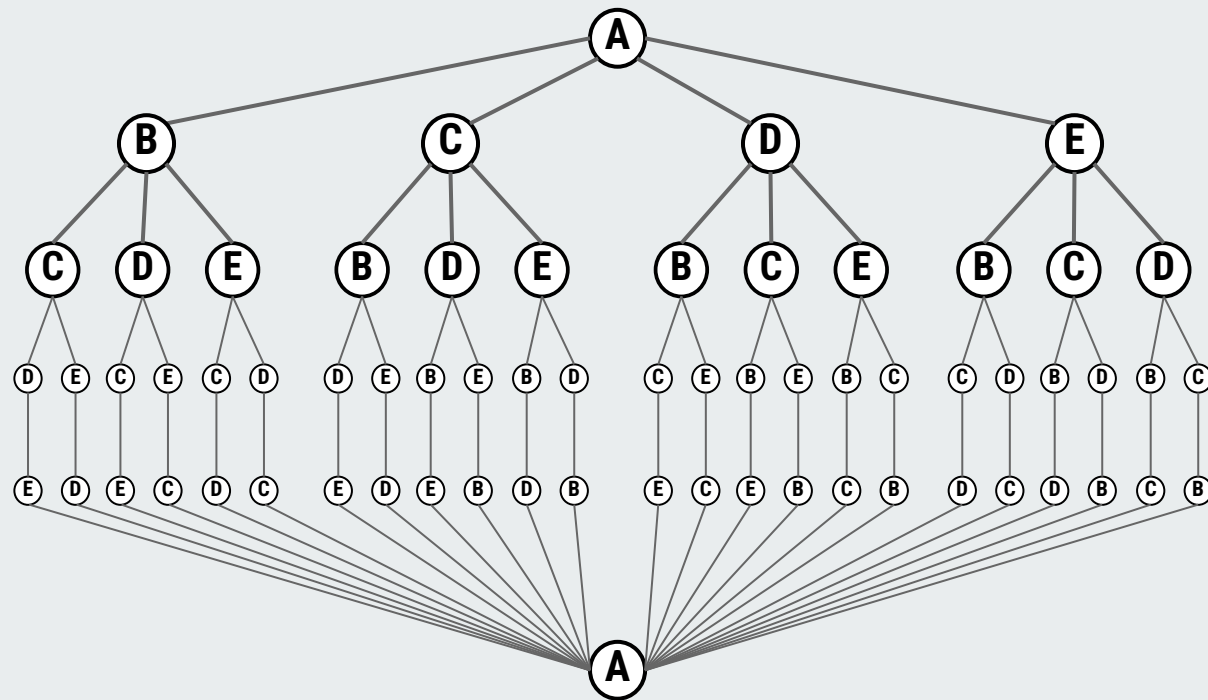
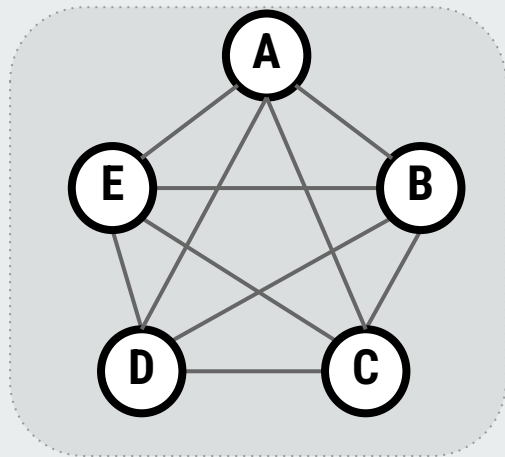


- ❑ Programação da rota de ônibus escolares (Angel et al., 1972).
- ❑ Programação de tripulação (Svestka & Huckfeldt, 1973).
- ❑ Fiação de computadores (Lenstra & Rinnooy Kan, 1974).
- ❑ Movimentação de materiais em depósitos (Ratliff & Rosenthal, 1983).
- ❑ Programação de transporte entre células de manufatura (Finke & Kusiak, 1985).
- ❑ Em grande parte dos problemas de roteamento de veículos (Bodin et al., 1983).
- ❑ Programação de operações de máquinas em manufatura (Kusiak & Finke, 1987).
- ❑ Otimização do movimento de ferramentas de corte (Chauny et al., 1987).
- ❑ Manutenção de motores de turbina a gás (Plante et al., 1987).
- ❑ Otimização de perfurações de furos em placas de circuitos impressos (Reinelt, 1989).
- ❑ Cristalografia através de raio X (Bland & Shallcross, 1989).
- ❑ Na solução de problemas de sequenciamento (Whitley et al., 1991).
- ❑ Agendamento de entrevistas (Gilbert & Hofstra, 1992).
- ❑ Trabalhos administrativos (Laporte et al., 1996).
- ❑ Na solução de problemas de programação e distribuição de tarefas em plantas (Salomon et al., 1997).
- ❑ Programação de laminação em siderurgia (Tang et al., 2000).

- ❑ Programação de operações de monitoramento de satélites (Czerniak, 2000).
- ❑ Construção de mapas genéticos e mapeamento de cromossomos (Agarwala et al., 2000).
- ❑ Programação de Impressão em gráficas ou jornais (Carter & Ragsdale, 2002).
- ❑ Planejamento do caminho de robôs cooperativos (Yu et al., 2002).
- ❑ Posicionamento de satélites para cobertura de comunicação (Saleh & Chelouah, 2004).
- ❑ Identificação do Alzheimer (De Vreese et al., 2005).
- ❑ Predição de propriedades de Proteínas (Johnson & Liu, 2006).
- ❑ Classificação de componentes eletrônicos (Goyal, 2006).
- ❑ Aplicações reais do TSP em Flowshop (Bagchi et al., 2006 e Caricato et al., 2007).
- ❑ Distribuição de combustível para postos de gasolina (Ismail & Ibrahim, 2008).
- ❑ Mamografia (Lupşa et al. 2008).
- ❑ Programação de laminação a frio (Zhao et al., 2011).
- ❑ Sistema antimíssil (Gao & Wang, 2011).
- ❑ Otimização da busca de planetas extra-solares (Kolemen & Kasdin, 2012).
- ❑ Perfuração de peças (Tsai et al., 2012)
- ❑ Cirurgia Endoscópica (Falcone et al., 2013).



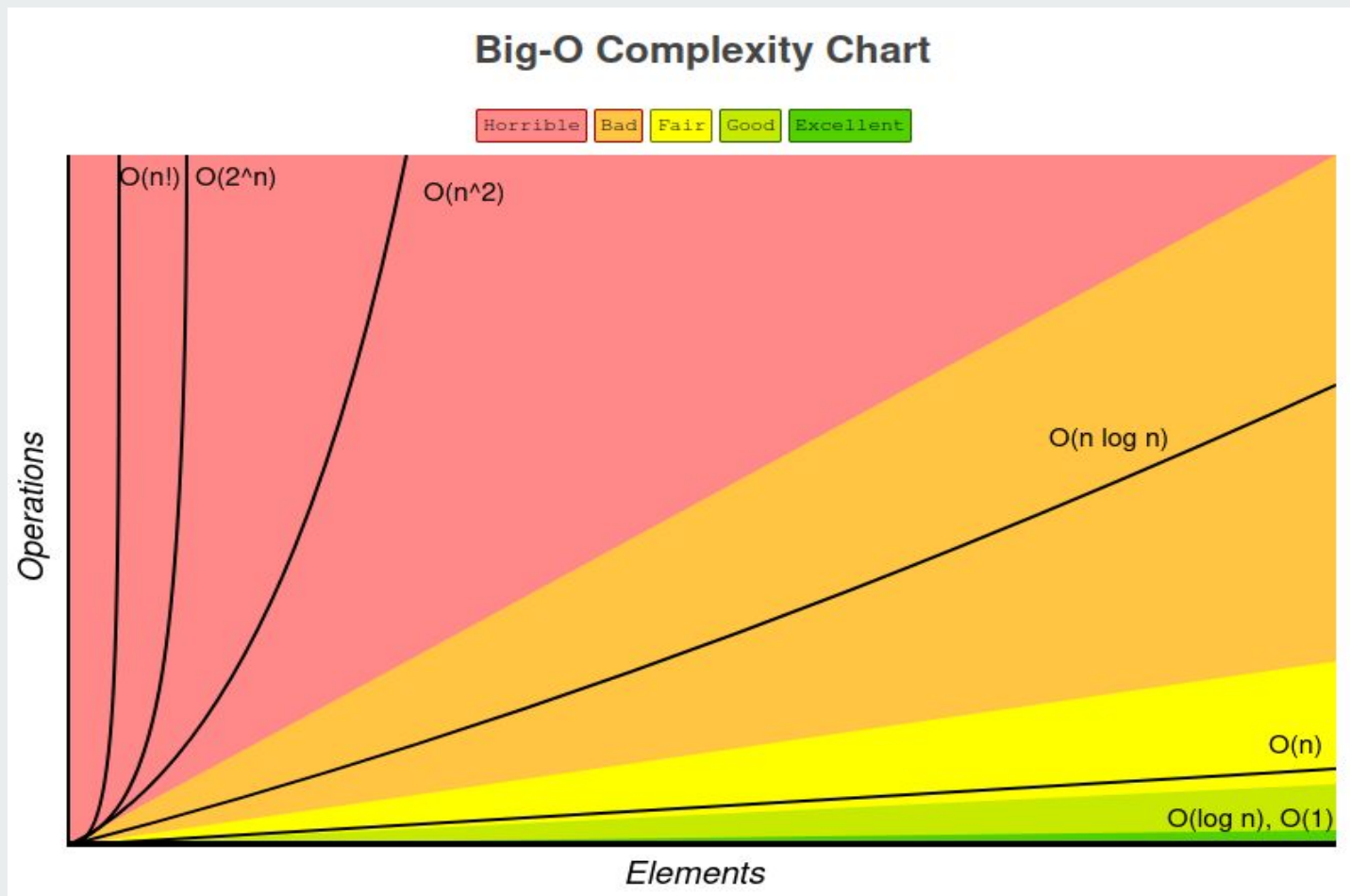
Para N cidades há $(N-1)!$ Rotas
 $(4-1)! = 3! = \mathbf{6 \text{ rotas}}$



Para N cidades há $(N-1)!$ Rotas
 $(5-1)! = 4! = \mathbf{24 \text{ rotas}}$

Para **N** cidades há **$(N-1)!$** Rotas.

N	# Rotas
7	720
8	5.040
9	40.320
10	362.880
11	3.628.800
12	39.916.800
26	15.511.210.043.330.985.984.000.000



Maior Crescimento



Menor Crescimento

$O(n!)$

fatorial

$O(2^n)$

exponencial

$O(n^3)$

cúbica

$O(n^2)$

quadrática

$O(n \log n)$

log linear

$O(n)$

linear

$O(\log n)$

logarítmica

$O(1)$

constante

$$O(1) = O(\text{yeah})$$

$$O(\log n) = O(\text{nice})$$

$$O(n) = O(k)$$

$$O(n^2) = O(\text{my})$$

$$O(2^n) = O(\text{no})$$

$$O(n!) = O(\text{mg})$$

$$O(n^n) = O(\text{sh*t!})$$

<https://www.bigocheatsheet.com/>

Função tempo/ complexidade	Quantidade de Dados: N				
	10	20	30	40	50
N	0,00001s	0,00002s	0,00003s	0,00004s	0,00005s
N^2	0,0001s	0,0004s	0,0009s	0,0016s	0,0036s
N^3	0,001s	0,008s	0,027s	0,064s	0,125s
2^N	0,001s	1,0s	17,19 min	12,7 dias	35,7 anos
3^N	0,059s	58 min	6,5 anos	3.855 séculos	200.000.000 séculos

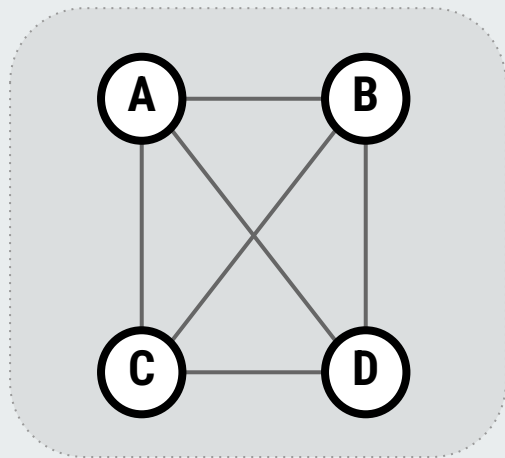
Obs: usando um computador capaz de realizar um milhão de operações aritméticas básicas por segundo.

Um problema **simétrico** com n cidades possui $(n-1)!/2$ soluções.

Se $n=20$, então existem mais de 10^{18} soluções.

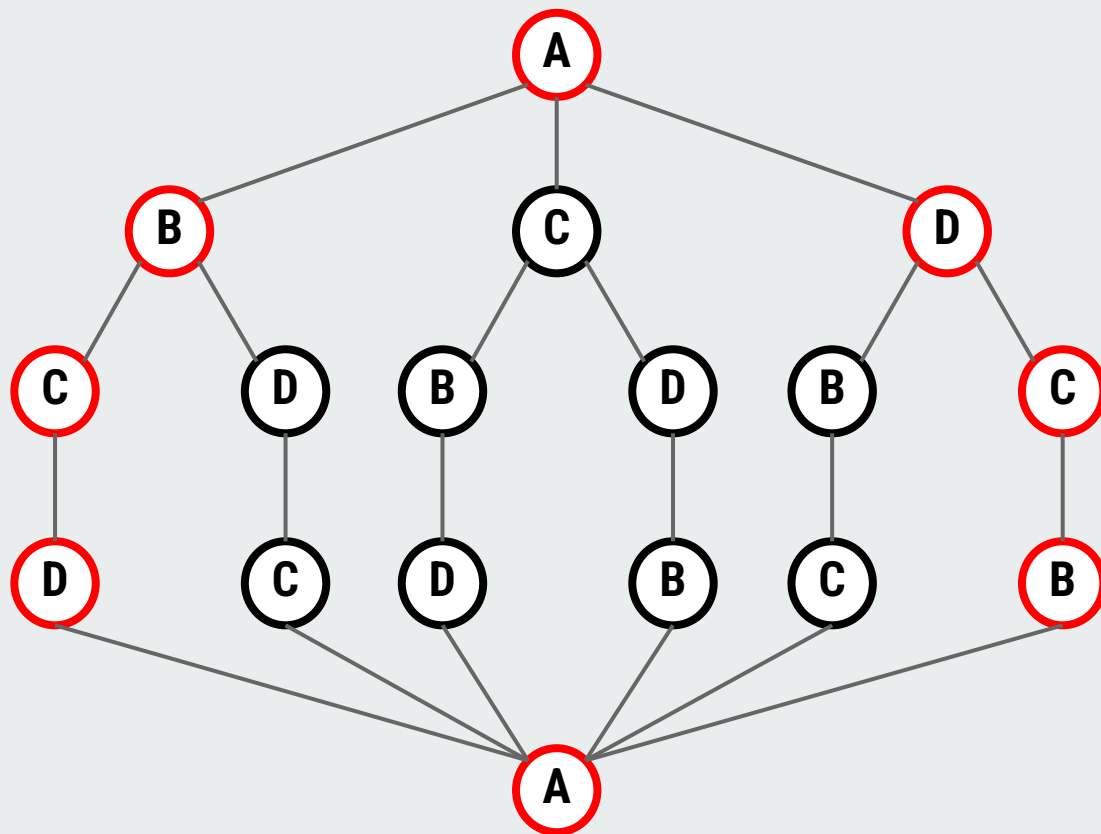
Karp (1975) provou que o problema pertence à classe NP-difícil.

O número de segundos desde o Big Bang até hoje é de aproximadamente $4,35 \times 10^{17}$ segundos



Para N cidades há $(N-1)!$ Rotas
 $(4-1)! = 3! = 6$ rotas

Para N cidades há $(N-1)!/2$ Rotas
3 rotas



Duas classes de algoritmos são usados para resolver problemas de otimização combinatória:

- *Algoritmos exatos.*
- *Técnicas heurísticas (ou algoritmos aproximados).*

Heurística do vizinho mais próximo (*Nearest Neighbor*)

- *O algoritmo do vizinho mais próximo é guloso (greedy):*
- *Começa de um vértice inicial **s**;*
- *Em cada passo, escolhe o vértice não visitado mais próximo;*
- *Repete até visitar todos os vértices;*
- *Retorna ao vértice inicial, formando um ciclo hamiltoniano;*
- *Não garante a solução ótima;*

Vizinho_mais_proximo(Grafo G , vértice s , matriz de distâncias d)

Para todo vértice $v \in V$ **faça**

$v.visitado = 0$;

insere ($percurso$, s)

$dist_total \leftarrow 0$

$v_atual \leftarrow s$

$v_atual.visitado = 1$

Enquanto houver vértice não visitado **faça**

$menor_dist \leftarrow \infty$

$prox \leftarrow \text{NULL}$

para todo vértice $w \in V$ tal que $w.visitado = 0$ **faça**

se $d[v_atual][w] < menor_dist$ **então**

$menor_dist \leftarrow d[v_atual][w]$

$prox \leftarrow w$

insere ($percurso$, $prox$)

$dist_total += menor_dist$

$v_atual \leftarrow prox$

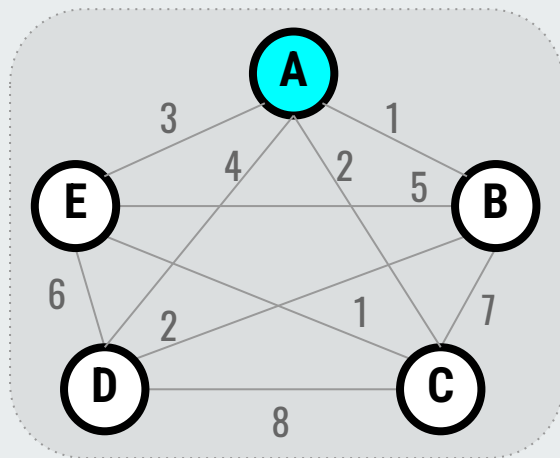
$v_atual.visitado \leftarrow 1$

// fecha o ciclo, voltando para o vértice inicial s

$dist_total += d[v_atual][s]$

insere ($percurso$, s)

Retorne ($percurso$, $dist_total$)



P=



dist =

d	A	B	C	D	E
A	-	1	2	4	3
B	1	-	7	2	5
C	2	7	-	8	1
D	4	2	8	-	6
E	3	5	1	6	-
visit					

Vizinho_mais_proximo(Grafo **G**, vértice **s**, matriz **d**)

Para todo vértice **v** $\in V$ **faça**

v.visitado = 0;

insere (**percurso**, **s**)

dist_total $\leftarrow 0$

v_atual $\leftarrow s$

v_atual.visitado = 1

Enquanto houver vértice não visitado **faça**

menor_dist $\leftarrow \infty$

prox $\leftarrow \text{NULL}$

para todo vértice **w** $\in V$ tal que **w**.visitado = 0 **faça**

se **d**[**v_atual**][**w**] < **menor_dist** **então**

menor_dist $\leftarrow d[v_atual][w]$

prox $\leftarrow w$

insere (**percurso**, **prox**)

dist_total += **menor_dist**

v_atual $\leftarrow \text{prox}$

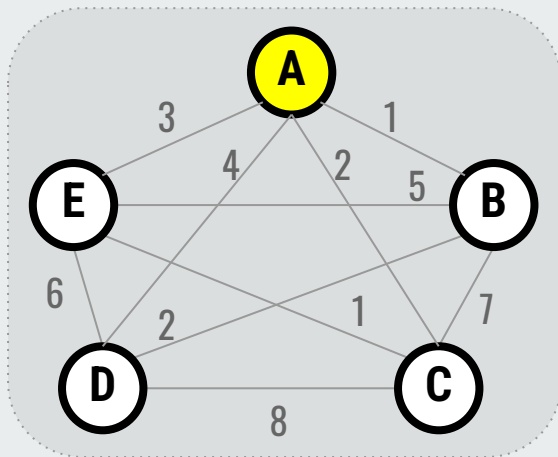
v_atual.visitado $\leftarrow 1$

// fecha o ciclo, voltando para o vértice inicial **s**

dist_total += **d**[**v_atual**][**s**]

insere (**percurso**, **s**)

Retorne (**percurso**, **dist_total**)



P=

A					
---	--	--	--	--	--

dist = 0

v_atual = A

menor_dist =

prox =

d	A	B	C	D	E
A	-	1	2	4	3
B	1	-	7	2	5
C	2	7	-	8	1
D	4	2	8	-	6
E	3	5	1	6	-
visit	1	0	0	0	0

Vizinho_mais_proximo(Grafo **G**, vértice **s**, matriz **d**)

Para todo vértice **v** $\in$ **V** **faça**

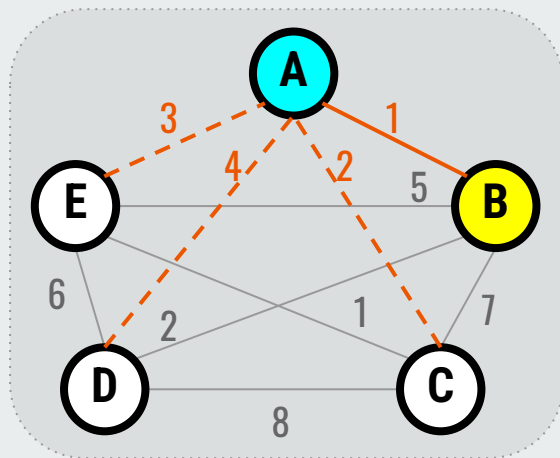
v.visitado = 0;

insere (**percurso**, **s**)

dist_total $\leftarrow$ 0

v_atual $\leftarrow$ **s**

v_atual.visitado = 1



P= A B

dist = 1

v_atual = A

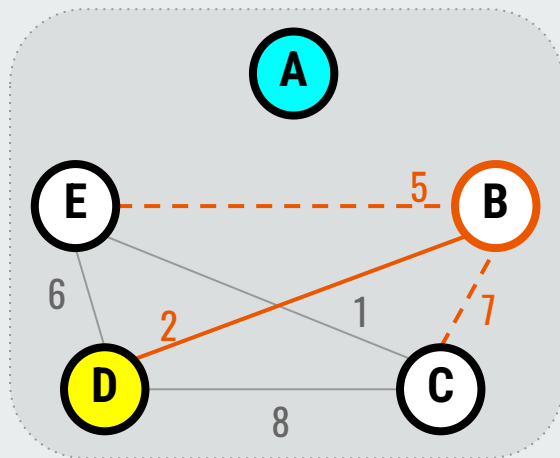
menor_dist = 1

prox = B

d	A	B	C	D	E
A	-	1	2	4	3
B	1	-	7	2	5
C	2	7	-	8	1
D	4	2	8	-	6
E	3	5	1	6	-
visit	1	1	0	0	0

```

Enquanto houver vértice não visitado faça
  menor_dist ← ∞
  prox ← NULL
  para todo vértice w ∈ V tal que w.visitado = 0 faça
    se d[v_atual][w] < menor_dist então
      menor_dist ← d[v_atual][w]
      prox ← w
  insere (percurso, prox)
  dist_total += menor_dist
  v_atual ← prox
  v_atual.visitado ← 1
  
```



P=

A	B	D			
---	---	---	--	--	--

$$\text{dist} = 1 + 2 = 3$$

$v_{\text{atual}} = B$

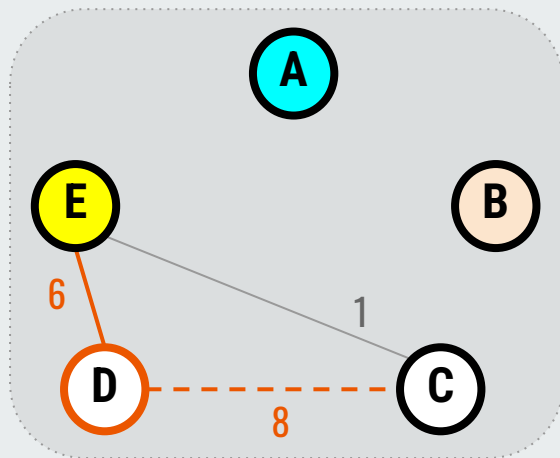
$\text{menor_dist} = 2$

$\text{prox} = D$

d	A	B	C	D	E
A	-	1	2	4	3
B	1	-	7	2	5
C	2	7	-	8	1
D	4	2	8	-	6
E	3	5	1	6	-
visit	1	1	0	1	0

```

Enquanto houver vértice não visitado faça
  menor_dist ← ∞
  prox ← NULL
  para todo vértice  $w \in V$  tal que  $w.\text{visitado} = 0$  faça
    se  $d[v_{\text{atual}}][w] < \text{menor\_dist}$  então
      menor_dist ←  $d[v_{\text{atual}}][w]$ 
      prox ←  $w$ 
  insere (percurso, prox)
  dist_total += menor_dist
   $v_{\text{atual}} \leftarrow \text{prox}$ 
   $v_{\text{atual}}.\text{visitado} \leftarrow 1$ 
  
```



P= A B D E

$$\text{dist} = 3 + 6 = 9$$

$v_{\text{atual}} = D$

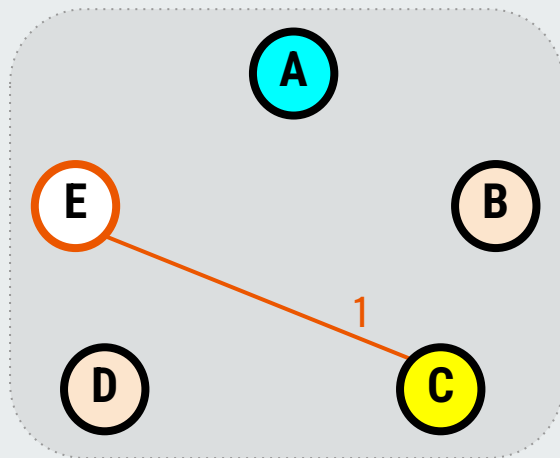
$\text{menor_dist} = 6$

$\text{prox} = E$

d	A	B	C	D	E
A	-	1	2	4	3
B	1	-	7	2	5
C	2	7	-	8	1
D	4	2	8	-	6
E	3	5	1	6	-
visit	1	1	0	1	1

```

Enquanto houver vértice não visitado faça
  menor_dist ← ∞
  prox ← NULL
  para todo vértice  $w \in V$  tal que  $w.\text{visitado} = 0$  faça
    se  $d[v_{\text{atual}}][w] < \text{menor\_dist}$  então
      menor_dist ←  $d[v_{\text{atual}}][w]$ 
      prox ←  $w$ 
  insere (percurso, prox)
  dist_total += menor_dist
   $v_{\text{atual}} \leftarrow \text{prox}$ 
   $v_{\text{atual}}.\text{visitado} \leftarrow 1$ 
  
```



P=

A	B	D	E	C	
---	---	---	---	---	--

$$\text{dist} = 9 + 1 = 10$$

$v_{\text{atual}} = \text{E}$

$\text{menor_dist} = 1$

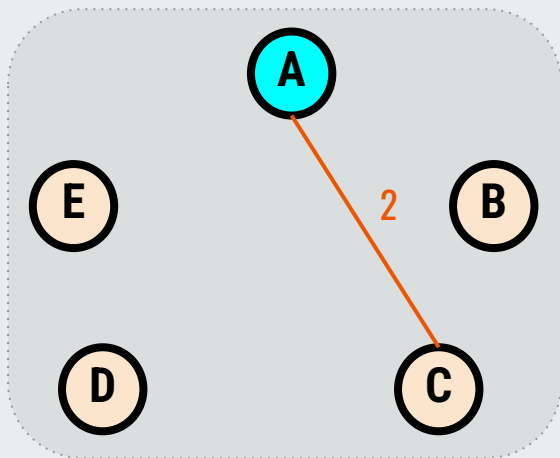
$\text{prox} = \text{C}$

d	A	B	C	D	E
A	-	1	2	4	3
B	1	-	7	2	5
C	2	7	-	8	1
D	4	2	8	-	6
E	3	5	1	6	-
visit	1	1	1	1	1

```

Enquanto houver vértice não visitado faça
  menor_dist ← ∞
  prox ← NULL
  para todo vértice  $w \in V$  tal que  $w.\text{visitado} = 0$  faça
    se  $d[v_{\text{atual}}][w] < \text{menor\_dist}$  então
      menor_dist ←  $d[v_{\text{atual}}][w]$ 
      prox ←  $w$ 
  insere (percurso, prox)
  dist_total += menor_dist
   $v_{\text{atual}} \leftarrow \text{prox}$ 
   $v_{\text{atual}}.\text{visitado} \leftarrow 1$ 

```



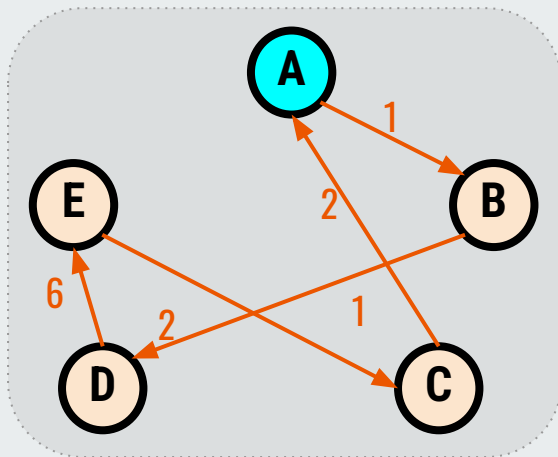
P=

A	B	D	E	C	A
---	---	---	---	---	---

$$\text{dist} = 10 + 2 = 12$$

d	A	B	C	D	E
A	-	1	2	4	3
B	1	-	7	2	5
C	2	7	-	8	1
D	4	2	8	-	6
E	3	5	1	6	-
visit	1	1	1	1	1

```
// fecha o ciclo, voltando para o vértice inicial s
dist_total += d[v_atual][s]
insere(percurso, s)
Retorne(percurso, dist_total)
```



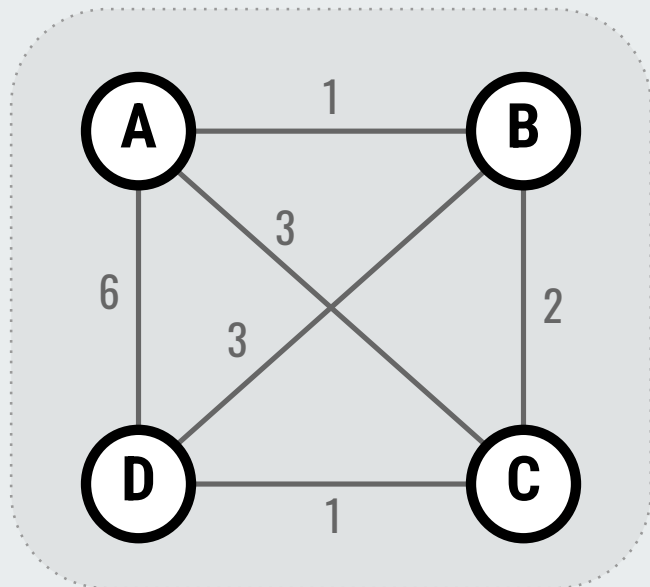
P=

A	B	D	E	C	A
---	---	---	---	---	---

dist = 12

d	A	B	C	D	E
A	-	1	2	4	3
B	1	-	7	2	5
C	2	7	-	8	1
D	4	2	8	-	6
E	3	5	1	6	-
visit	1	1	1	1	1

```
// fecha o ciclo, voltando para o vértice inicial s
dist_total += d[v_atual][s]
insere(percursos, s)
Retorne(percursos, dist_total)
```



Observe que:

Iniciando no vértice **A**:

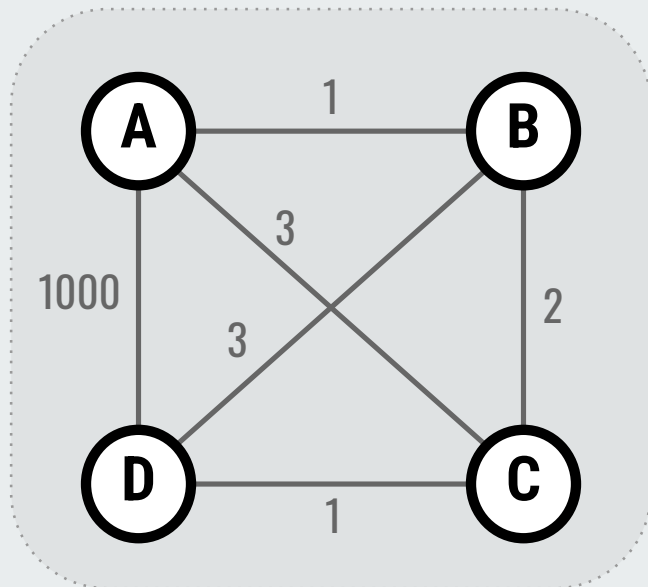
$A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$

Custo = 10

Melhor solução:

$A \rightarrow B \rightarrow D \rightarrow C \rightarrow A$

Custo = 8



Observe que:

Iniciando no vértice **A**:

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$

Custo = 1004

Melhor solução:

$A \rightarrow B \rightarrow D \rightarrow C \rightarrow A$

Custo = 8

COMPLEXIDADE?

Vizinho_mais_proximo(Grafo G , vértice s , matriz de distâncias d)

Para todo vértice $v \in V$ **faça**

v .visitado = 0;

insere (**percurso**, s)

$dist_total \leftarrow 0$

$v_atual \leftarrow s$

v_atual .visitado = 1

Enquanto houver vértice não visitado **faça**

$menor_dist \leftarrow \infty$

$prox \leftarrow \text{NULL}$

para todo vértice $w \in V$ tal que w .visitado = 0 **faça**

se $d[v_atual][w] < menor_dist$ **então**

$menor_dist \leftarrow d[v_atual][w]$

$prox \leftarrow w$

insere (**percurso**, $prox$)

$dist_total += menor_dist$

$v_atual \leftarrow prox$

v_atual .visitado $\leftarrow 1$

// fecha o ciclo, voltando para o vértice inicial s

$dist_total += d[v_atual][s]$

insere (**percurso**, s)

Retorne (**percurso**, $dist_total$)

Heurística do vizinho mais próximo (*Nearest Neighbor*)

- *Complexidade: $O(n^2)$*
- *A solução gerada depende muito do vértice inicial.*
- *Aplicar a heurística começando em cada vértice e ficar com a melhor solução: $O(n^3)$*

Vamos programar?



1. Implemente o Método do vizinho mais próximo, de forma a obter um percurso (ciclo) e o custo deste ciclo, a partir de um vértice inicial **s**.



Dúvidas?!

